

Table des matières

1 Introduction à la Théorie des Graphes 4

1.1 Définition et historique 4

1.2 Exemples concrets simples 4

2 Définitions et Terminologie 4

2.1 Définitions de base 4

2.2 Types de graphes 4

2.2.1 Graphe non orienté 4

2.2.2 Graphe orienté (digraphe) 4

2.2.3 Graphe pondéré 5

2.3 Terminologie avancée 5

3 Représentation des Graphes 5

3.1 Matrice d'adjacence 5

3.2 Liste d'adjacence 5

3.2.1 Comparaison des représentations 6

4 Parcours de Graphes 6

4.1 Parcours en Profondeur (DFS) 6

4.1.1 Exemple d'exécution 6

4.2 Parcours en Largeur (BFS) 6

4.2.1 Exemple d'exécution 7

5 Algorithmes de Plus Court Chemin 7

5.1 Algorithme de Dijkstra 7

5.1.1 Exemple étape par étape 7

5.2 Complexité 8

6 Algorithmes d'Arbre Couvrant 8

6.1 Algorithme de Kruskal 8

6.2 Algorithme de Prim 8

7 Coloration de Graphes 8

7.1 Problème de coloration 8

7.1.1 Algorithme glouton 8



```
[12pt,a4paper]article [T1]fontenc [utf8]inputenc [french]babel amsmath,amssymb tikz tkz-graph
tkz-berge algorithm algorithmic enumitem hyperref geometry listings xcolor caption
margin=2.5cm
```

Cours Complet de Théorie des Graphes

Table des matières

Introduction à la Théorie des Graphes

Définition et historique

La **théorie des graphes** est une branche des mathématiques discrètes qui étudie les graphes, structures composées d'objets (sommets) reliés par des liens (arêtes).

Historique :

- 1736 : Leonhard Euler résout le problème des sept ponts de Königsberg
- 1852 : Francis Guthrie pose le problème de la coloration des cartes
- 20ème siècle : Développement avec l'informatique

Exemples concrets simples

- **Réseau social** : Personnes (sommets) et amitiés (arêtes)
- **Réseau routier** : Villes (sommets) et routes (arêtes)
- **Internet** : Routeurs (sommets) et connexions (arêtes)
- **Circuit électrique** : Composants (sommets) et fils (arêtes)

Définitions et Terminologie

Définitions de base

- **Graphe** $G = (V, E)$ où V est l'ensemble des sommets et E l'ensemble des arêtes
- **Ordre d'un graphe** : Nombre de sommets $|V|$
- **Taille d'un graphe** : Nombre d'arêtes $|E|$

Types de graphes

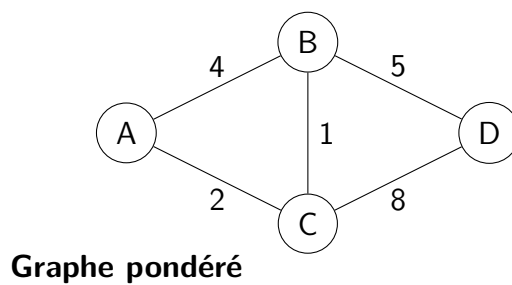
Graphe non orienté

Graphe non orienté simple

Graphe orienté (digraphe)

Graphe orienté

Graphe pondéré



Terminologie avancée

- **Degré d'un sommet** : Nombre d'arêtes incidentes
- **Chemin** : Séquence de sommets connectés
- **Cycle** : Chemin qui commence et termine au même sommet
- **Graphe connexe** : Toute paire de sommets reliée par un chemin
- **Graphe complet** K_n : Tous les sommets sont reliés deux à deux

Représentation des Graphes

Matrice d'adjacence

Pour un graphe $G = (V, E)$ avec $n = |V|$, la matrice d'adjacence A est une matrice $n \times n$ où :

$$A[i][j] = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 0 & \text{sinon} \end{cases}$$

$$A = \begin{bmatrix} & B & C & D \\ A & & & \\ B & & & \\ C & & & \\ D & & & \end{bmatrix}$$

Matrice d'adjacence

Liste d'adjacence

Pour chaque sommet, on stocke la liste de ses voisins :

- A : [B, C]
- B : [A, C, D]

- C : [A, B, D]
- D : [B, C]

Comparaison des représentations

Opération	Matrice	Liste
Test d'adjacence	$O(1)$	$O(d)$
Ajout d'arête	$O(1)$	$O(1)$
Suppression d'arête	$O(1)$	$O(d)$
Mémoire	$O(n^2)$	$O(n+m)$

Complexités comparées (n sommets, m arêtes, d degré)

Parcours de Graphes

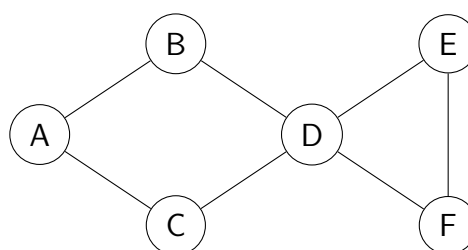
Parcours en Profondeur (DFS)

Idée : Explorer aussi loin que possible avant de revenir en arrière.

Graphes G , sommet s Liste des sommets visités
 marquer s comme visité
 traiter s chaque voisin v de s
 v non visité DFS(G, v)

Graphes G , sommet s Liste des sommets visités
 créer une pile P empiler s sur P
 P non vide $v = \text{dépiler}$
 P v non visité marquer v comme visité
 traiter v chaque voisin w de v w non visité empiler w sur P

Exemple d'exécution



DFS depuis A : $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

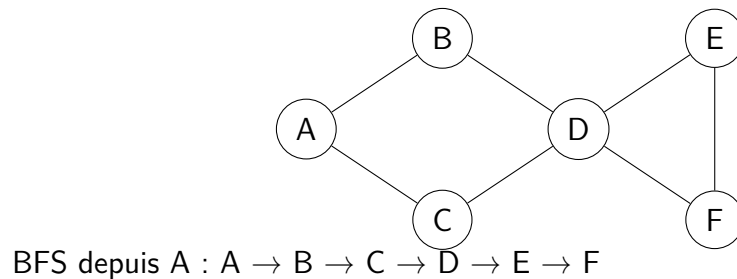
Parcours en Largeur (BFS)

Idée : Explorer tous les voisins avant de passer au niveau suivant.

Graphes G , sommet s Liste des sommets visités par niveau
 créer une file F enfile s dans F marquer s comme visité
 F non vide $v = \text{défiler}$ F traiter v chaque voisin w de v w non visité marquer w comme

visité enfile w dans F

Exemple d'exécution



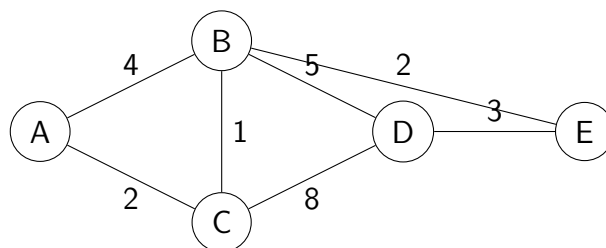
Algorithmes de Plus Court Chemin

Algorithme de Dijkstra

Problème : Trouver le plus court chemin entre un sommet source et tous les autres dans un graphe pondéré avec poids positifs.

Graphe G , source s Distances minimales et prédécesseurs $\text{dist}[v] \leftarrow \infty$ pour tout $v \in V$ $\text{dist}[s] \leftarrow 0$
 $\text{prev}[v] \leftarrow \text{null}$ pour tout $v \in V$ créer une file de priorité Q contenant tous les sommets Q non vide
 $u \leftarrow$ sommet de Q avec dist minimale retirer u de Q chaque voisin v de u $\text{alt} \leftarrow \text{dist}[u] + \text{poids}(u, v)$
 $\text{alt} < \text{dist}[v]$ $\text{dist}[v] \leftarrow \text{alt}$ $\text{prev}[v] \leftarrow u$ dist, prev

Exemple étape par étape



1. Initialisation : $\text{dist}[A]=0$, autres =
2. A choisi : $\text{dist}[B]=4$, $\text{dist}[C]=2$
3. C choisi : $\text{dist}[D]=\min(, 2+8=10)=10$
4. B choisi : $\text{dist}[D]=\min(10, 4+5=9)=9$, $\text{dist}[E]=4+2=6$
5. D choisi : $\text{dist}[E]=\min(6, 9+3=12)=6$

Complexité

- Matrice d'adjacence : $O(|V|^2)$
- Liste d'adjacence + tas : $O((|V| + |E|) \log |V|)$

Algorithmes d'Arbre Couvrant

Algorithme de Kruskal

Trouver l'arbre couvrant de poids minimal.

Graphe connexe pondéré G Arbre couvrant minimal
Trier les arêtes par poids croissant
initialiser une forêt F (chaque sommet est un arbre)
résultat = ensemble vide
chaque arête (u,v) par ordre croissant
 u et v dans des arbres différents
ajouter (u,v) au résultat
fusionner les arbres de u et v

Algorithme de Prim

Graphe connexe pondéré G Arbre couvrant minimal
Choisir un sommet initial arbitraire
initialiser un arbre T avec ce sommet
 T n'a pas tous les sommets
trouver l'arête de poids minimum entre T et $V-T$
ajouter cette arête et son extrémité à T

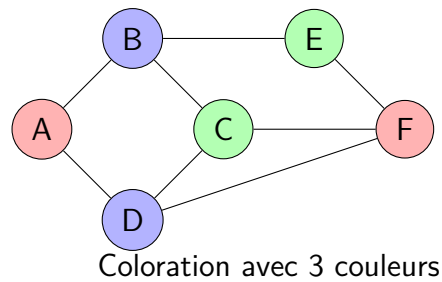
Coloration de Graphes

Problème de coloration

Définition : Colorier les sommets d'un graphe avec un nombre minimum de couleurs tel que deux sommets adjacents n'aient pas la même couleur.

Algorithme glouton

Graphe G Coloration des sommets
Ordonner les sommets (par degré décroissant)
couleurs disponibles = $1, 2, \dots, n$
chaque sommet v dans l'ordre
couleurs interdites = couleurs des voisins colorés
attribuer à v la plus petite couleur non interdite



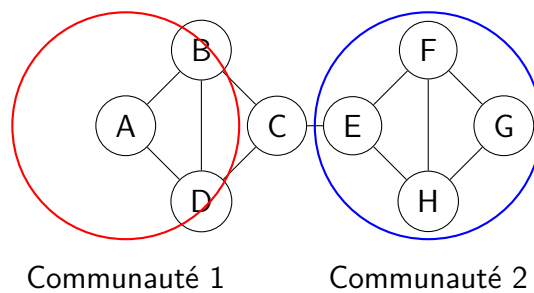
Applications Concrètes

Réseaux Sociaux

Problème : Trouver les communautés, influencerurs, diffusion d'information.

Solution :

- Analyse des composantes connexes
- Centralité des sommets (degré, intermédiarité, proximité)
- Recherche de cliques (groupes complets)

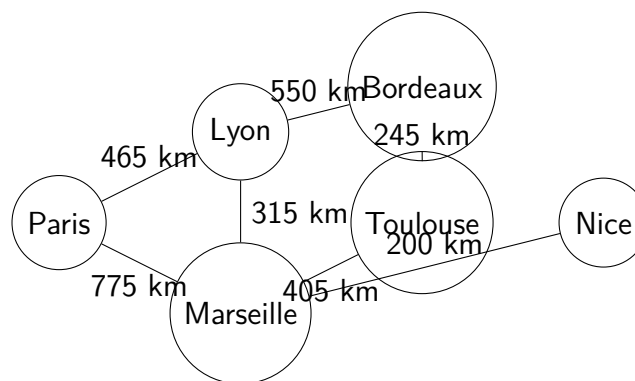


Systèmes de Navigation GPS

Problème : Calculer le chemin le plus rapide entre deux points.

Solution : Algorithme de Dijkstra ou A* sur le réseau routier.

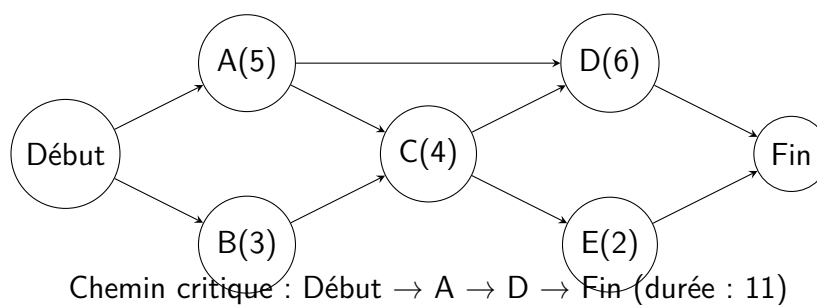
Exemple de réseau routier simplifié



Planification de Projet (PERT/CPM)

Problème : Organiser les tâches d'un projet pour minimiser la durée totale.

Solution : Graphe d'ordonnancement avec tâches comme sommets et contraintes comme arêtes.



Réseaux de Télécommunications

Problème : Optimiser le routage des données, éviter les congestions.

Solution : Algorithmes de flux maximum (Ford-Fulkerson).

Valeur Ajoutée dans l'Industrie

Logistique et Transport

- **Problème** : Optimisation des tournées de livraison (VRP)
- **Solution** : Algorithmes sur graphes + heuristiques
- **Valeur** : Réduction de 15-30% des coûts logistiques

E-commerce et Recommandations

- **Problème** : Recommander des produits pertinents
- **Solution** : Graphes de co-achat, filtrage collaboratif
- **Valeur** : Augmentation du taux de conversion de 5-20%

Détection de Fraude

- **Problème** : Identifier des comportements suspects
- **Solution** : Analyse des graphes de transactions
- **Valeur** : Réduction des pertes de 20-50%

Bio-informatique

- **Problème** : Comprendre les interactions protéiques
- **Solution** : Graphes d'interaction protéine-protéine
- **Valeur** : Accélération de la recherche médicale

Implémentation en Python

Listing 1 – Structure de base d'un graphe

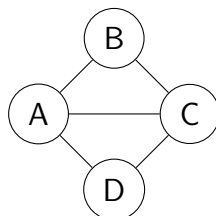
```
1 class Graphe:
2     def __init__(self, oriente=False):
3         self.graphe = {}
4         self.oriente = oriente
5
6     def ajouter_sommet(self, sommet):
7         if sommet not in self.graphe:
8             self.graphe[sommet] = []
9
10    def ajouter_arete(self, u, v, poids=1):
11        self.ajouter_sommet(u)
12        self.ajouter_sommet(v)
13        self.graphe[u].append((v, poids))
14        if not self.oriente:
15            self.graphe[v].append((u, poids))
16
17    def bfs(self, depart):
18        from collections import deque
19        visites = set([depart])
```

```
20     file = deque([depart])
21     ordre = []
22
23     while file:
24         sommet = file.popleft()
25         ordre.append(sommet)
26         for voisin, _ in self.graphe[sommet]:
27             if voisin not in visites:
28                 visites.add(voisin)
29                 file.append(voisin)
30     return ordre
31
32 def dijkstra(self, depart):
33     import heapq
34     distances = {sommet: float('inf') for sommet in self.graphe}
35     distances[depart] = 0
36     file_priorite = [(0, depart)]
37
38     while file_priorite:
39         dist_actuelle, sommet = heapq.heappop(file_priorite)
40
41         if dist_actuelle > distances[sommet]:
42             continue
43
44         for voisin, poids in self.graphe[sommet]:
45             distance = dist_actuelle + poids
46             if distance < distances[voisin]:
47                 distances[voisin] = distance
48                 heapq.heappush(file_priorite, (distance, voisin))
49
50     return distances
```

Exercices Pratiques

Exercice 1 : Représentation

Soit le graphe suivant :

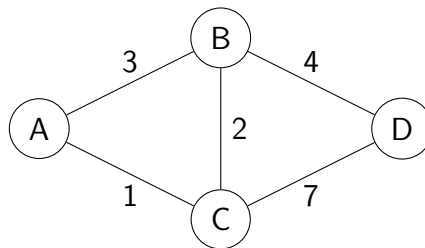


1. Donner la matrice d'adjacence de ce graphe. 2. Donner la liste d'adjacence. 3. Quel est le degré de chaque sommet ?

Exercice 2 : Parcours

Pour le graphe précédent : 1. Donner un ordre possible de parcours en profondeur depuis A. 2. Donner un ordre possible de parcours en largeur depuis A.

Exercice 3 : Plus court chemin



Calculer les plus courts chemins depuis A vers tous les autres sommets en utilisant l'algorithme de Dijkstra.

Conclusion

La théorie des graphes est un domaine fondamental en informatique et mathématiques appliquées. Ses applications sont vastes et touchent presque tous les secteurs de l'industrie moderne.

Points clés à retenir

- Les graphes modélisent efficacement les relations et connexions
- Les algorithmes de parcours (BFS/DFS) sont la base de nombreux autres algorithmes
- L'algorithme de Dijkstra résout efficacement le problème du plus court chemin avec poids positifs
- La théorie des graphes apporte une valeur ajoutée significative dans l'optimisation, l'analyse de réseaux et la prise de décision

Perspectives futures

- Intelligence Artificielle : Apprentissage sur graphes (Graph Neural Networks)
- Big Data : Analyse de graphes à grande échelle

— Sécurité : Détection d'anomalies dans les réseaux

**Fin du cours sur
la théorie des graphes**